AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_vhdl_uppgift_0.pdf Version : V1.0	

TEIS ECS - Embedded Computer System -

Darwin Rengifo

2018-10-09

Sammanfattning:
Det är en prototyp av automatisk dörr-styrning som ska konstruerar i två varianter. En enkla dörr-styrningen med både synkrona utgångar (Mealy+Moore), en Mealy maskin och en Moore maskin med asynkron reset och en avancerade lås dörr-styrningen med synkrona utgångar (Mealy+Moore). Alla fyra prototyper kommer att validerar på FPGA-Kortet DE10-Lite.

Innehåll

1. INLEDNING 4

2. KRAVSPECIFIKATION 4

3. BESKRIVNINGEN AV DE OLIKA TILLSTÅNDMASKINERNA FÖR DEN ENKLA DÖRRSTYRNINGEN 6

3.1 MEALY - UTAN LÅS 6

3.1.1 SYMBOL 7

3.1.2 SYSTEM ARKITEKTUR 7

3.1.3 BESKRIVNING AV TILLSTÅNDGRAF 7

3.1.4 MODELSIM 8

3.1.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN 8

3.2 MOORE - UTAN LÅS 9

3.2.1 SYMBOL 9

3.2.2 SYSTEM ARKITEKTUR 10

3.2.3 BESKRIVNING AV TILLSTÅNDGRAF 10

3.2.4 MODELSIM 10

3.2.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN 11

3.3 MEALY + MOORE - UTAN LÅS 12

3.3.1 SYMBOL 12

3.3.2 SYSTEM ARKITEKTUR 12

3.3.3 BESKRIVNING AV TILLSTÅNDGRAF 13

3.3.4 MODELSIM 13

3.3.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN 13

4. BESKRIVNINGEN AV TILLSTÅNDMASKINEN FÖR DEN AVANCERAD DÖRRSTYRNINGEN MED LÅS 15

4.1 MEALY + MOORE - MED LÅS 15

4.1.1 SYMBOL 15

4.1.2 SYSTEM ARKITEKTUR 15

4.1.3 BESKRIVNING AV TILLSTÅNDGRAF 16

4.1.4 MODELSIM 16

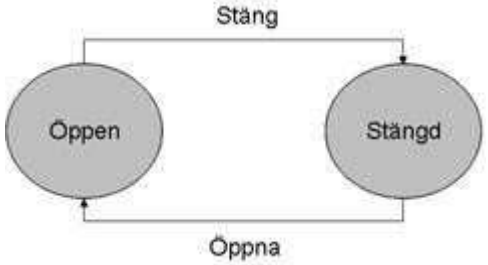
4.1.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN 17

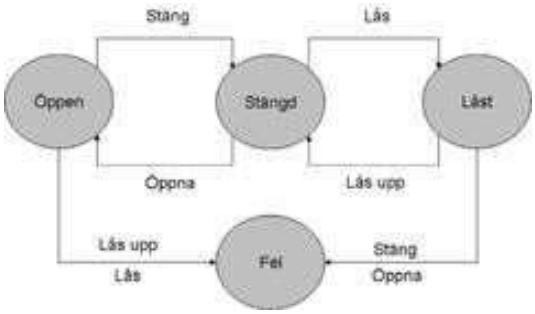
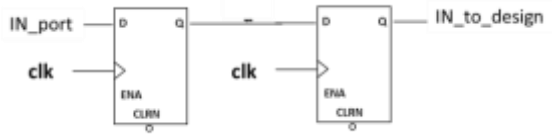
1. INLEDNING

Denna rapport beskriver de olika typer av tillståndsmaskiner skrivet i VHDL. Systemet har analyserats genom simulering och verifiering i ModelSim och därefter validerats på ett lämpligt FPGA-kort.

2. KRAVSPECIFIKATION

Tabell 1. Kravspecifikation från TEIS vhdl_2_uppgift_0

Krav id	Beskrivning	Utfört Ja/nej
	Konstruktionskrav	
1	Skapa ett Quartus-projekt med namnet ”uppgift_vhdl_0_xx”, där ”xx” är maskinens namn.	Ja
2a	Dörr utan lås Skapa tre VHDL-projekt (tre olika projekt med en VHDL-fil i varje). 1) Två projekt, en Mealy och en Moore maskin med asynkrona utgångar. 2) Ett projekt med en ”Mealy+Moore med synkrona utgångar”. Synkron utgång betyder att utgången kommer från vippans utgång (se PowerPoint i teori 4b). Om det funnits logik efter, hade det inneburit en viss fördröjning (”glitches” skapas). Med synkrona utgångar finns inga ”glitches”. Mera information i slutet av uppgiften. Den enkla dörrstyrningen ska ske med två tillstånd, se figur 2. Insignaler: KEY(0) = nedtryckt; Stäng KEY(1) = nedtryckt; Öppna Reset_n och CLOCK50 Utsignaler: LEDR[0] = Lyser när dörren är Öppen LEDR[1] = Lyser när dörren är stängd Funktion: Starttillstånd Öppen dörr (efter reset_n)	Ja
	 **Figur 2. Enkel dörr med två tillstånd - tillståndsgraf**	Ja

2b	Dörr med lås Skapa en VHDL-fil med valfri tillståndsmaskin (kan byggas vidare på ett av de gamla projekten, se figur 3). Det finns ett feltillstånd som man bara kan komma ifrån med att aktivera signalen reset_n (Switch 9). Insignaler (eller brytare): KEY(0) = nedtryckt; Stäng KEY(1) = nedtryckt; Öppna KEY(2) = nedtryckt; Lås. KEY(3) = nedtryckt; Lås upp. SW(9) = Reset_n och CLK_50 = CLOCK50 Utsignaler: LEDR[0] = Lyser när dörren är Öppen LEDR[1] = Lyser när dörren är stängd LEDR[2] = Lyser när dörren är låst LEDR[9] = Lyser när dörren har kommit i feltillståndet Funktion: Starttillstånd: Öppen dörr, efter reset_n  **Figur 3: Avancerad dörr - tillståndsgraf, starttillstånd öppen dörr**	Ja
2c	Valfria kontakter och lysdioder på FPGA-kortet kan användas.	Ja
	Testprotokoll	
3	Inga krav	
	VHDL-kod	
5a	Regler och riktlinjer för VHDL och C: regel nr 1 och nr 3.	
5b	Lägg in två vippor i rad på alla insignaler utom på klockan (clk i figuren). (Varför kommer att diskuteras senare i kursen).	Ja
	 **Figur 4: Två vippor i rad på ingångarna**	
	Verifierings-/Valideringskrav	
6	Simulera några tillstånd och validera på kortet.	Ja

Rapport		
7a	1. Beskriv de olika tillståndsmaskinerna för den enkla dörrstyrningen a. Beskriv med tillståndsgraf* b. Visa med pulsdiagram från ModelSim när maskinen går mellan några tillstånd. c. Visa med grindar (RTL från Quartus verktyget). d. Analysera sambandet mellan grindnivå och VHDL-koden. T.ex. var finns tillstånden sparade, nästa tillstånd, insignaler, utsignaler med mera. 2. Beskriv tillståndsmaskinerna för dörrstyrningen med lås a. Generera från Quartus tillståndsgraf (fungerar inte på för små maskiner) b. Analysera som ovan	Ja
Leveranskrav		
8	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en fil (zip eller rar): • Rapport (se rapportkrav) • VHDL-filerna för de olika tillståndsmaskinerna (Totalt 4 st.) Namnet på leveransen ska vara ”fornamn_efternamn_vhdl_uppgift_0”..	Ja

*) De små maskinerna kan inte automatiskt genereras med Quartus, för det behövs större tillståndsmaskiner.

3. BESKRIVNINGEN AV DE OLIKA TILLSTÅNDMASKINERNA FÖR DEN ENKLA DÖRRSTYRNINGEN

Den enkla dörrstyrningen är varit konstruerat på tre olika prototyper av tillständmaskiner som är Mealy, Moore och Mealy+Moore. På nästa stegen det visar beskrivningarna av varje prototyp.

3.1 MEALY - UTAN LÅS

Den här prototyp är en enkel variant, som består av fyra ingången signaler och ett utgång signal som kontrollerar funktionaliteten av den dörr konstruerade med Mealy maskin.

Ingången signaler

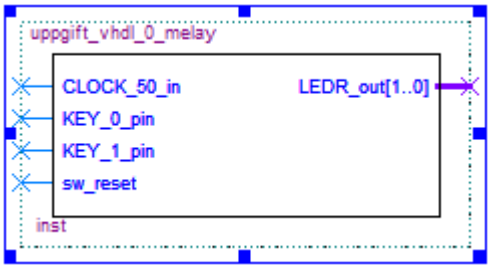
CLOCK_50_IN – (std_logic) : Den klock signal av 50MHz är kopplat till CLOCK_50.
KEY_0_PIN – (std_logic) : Ändra till nästa tillståndmaskin stäng.
KEY_1_PIN – (std_logic) : Ändra till nästa tillståndmaskin oppen.
sw_reset – (std_logic) : Reset av signalen. Active low

Utgång signal :

LEDR_out[1 downto 0] : Signal som visar ändringar av tillståndsmaskiner.

3.1.1 SYMBOL

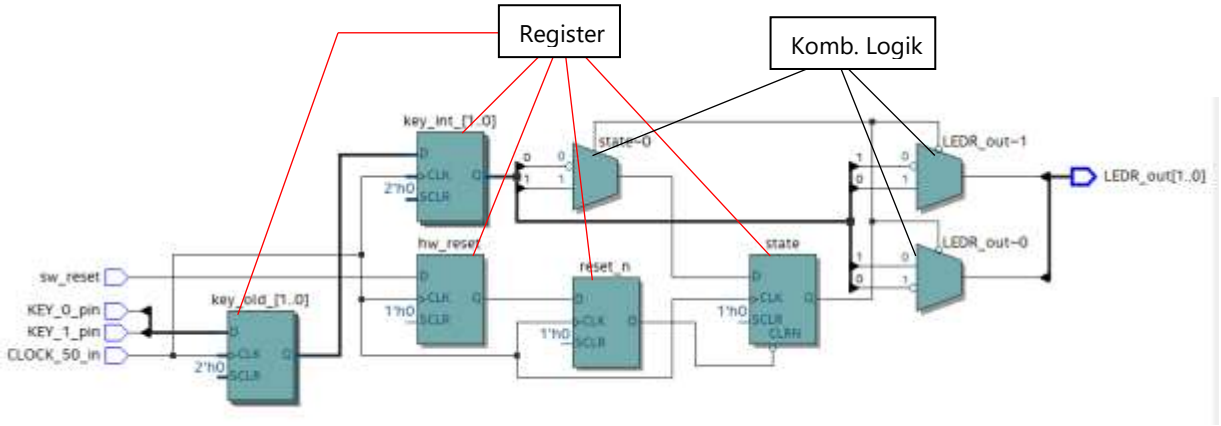
Ingångarna och utgångar av tillståndsmaskin Mealy visas i Figur 1.



Figur 1. Mealy tillståndsmaskin symbol

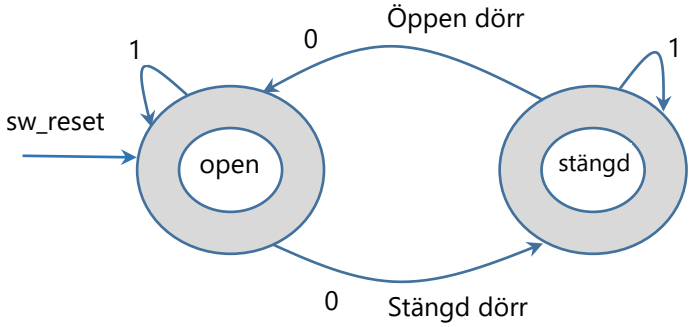
3.1.2 SYSTEM ARKITEKTUR

Den tillståndsmaskin Mealy innehålla en kombinatoriska logik och register state. Det kan se i figur 2 att utgångarna är en funktion av nuvarande tillstånd och samtliga ingångar.



Figur 2. Mealy system arkitektur

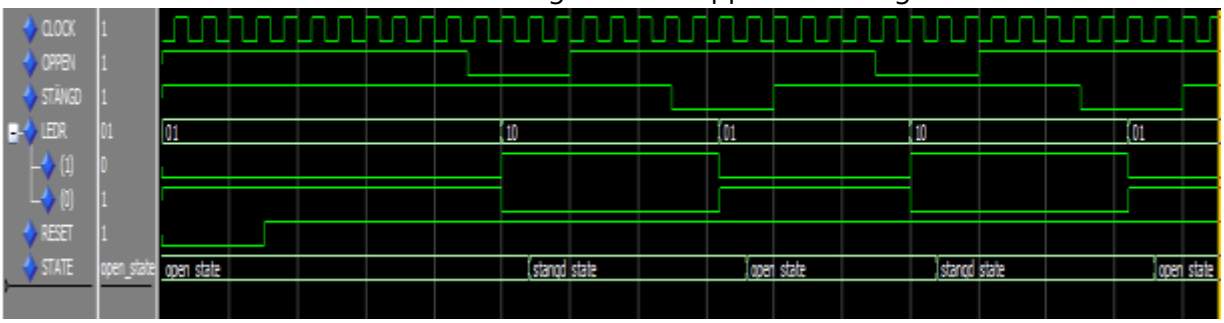
3.1.3 BESKRIVNING AV TILLSTÅNDGRAF



Figur 3. Mealy tillståndgraf

3.1.4 MODELSIM

ModelSim visar när maskinen går mellan öppen och stängd tillstånd.



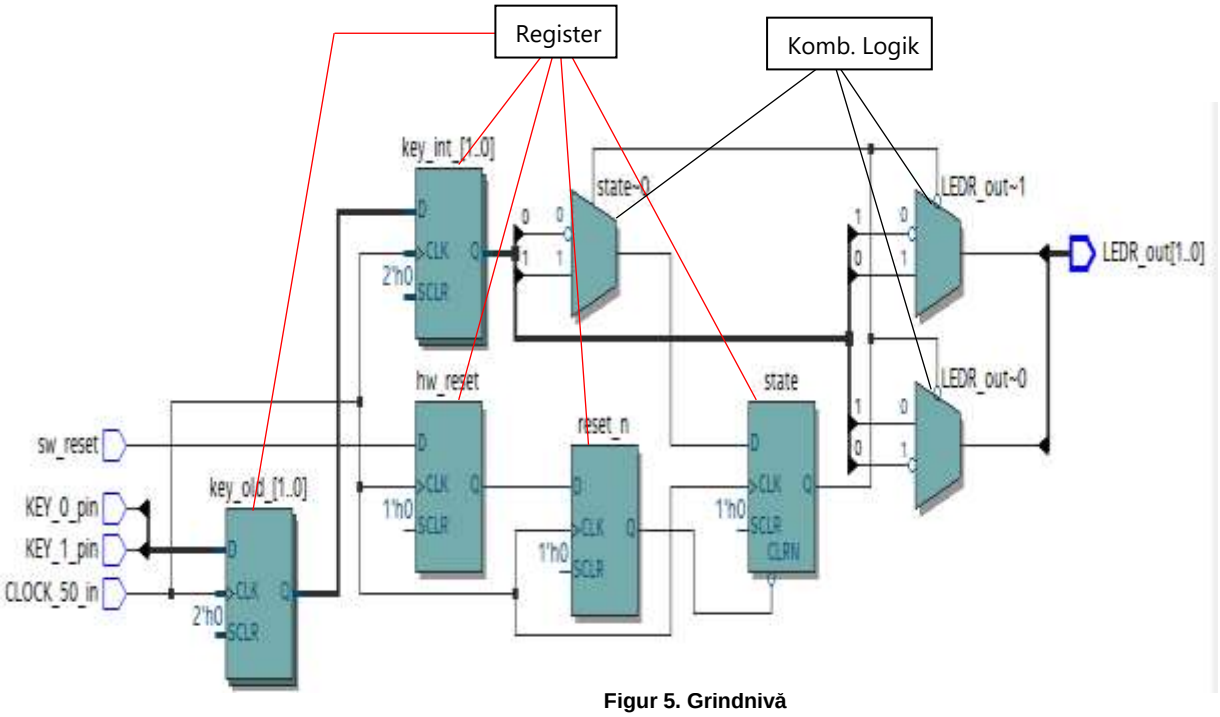
Figur 4. Pulsdiagram från ModelSim. Asynkron utgång.

3.1.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN

I figur 5 och tabell 2 visar sambandet mellan grindnivå och vhdl-koden, det står i vhdl-koden två processor som beskriver funktionaliteten av Mealy maskinen.

Registers: Den delen det fungerar för att behåller nuvarande tillstånd(state)

Kombinatorisk Logic: Den delen visar hur utgångarna är en funktion av nuvarande tillstånd(state) och samtliga ingångar(key_in_0, key_in_1).



Figur 5. Grindnivå

Tabell 2. Mealy maskin. Sambandet mellan grindnivå och vhdl-koden

Resister	Kombinatorisk Logic

```
process(CLOCK_50_in, reset_n)
begin

   if reset_n = '0' then
   -- Clear register, and clear state
      state <= open_state;

   elsif rising_edge(CLOCK_50_in) then
      case state is
         ---- Statemachine ----
         when open_state =>

            if key_int_0 ='0' then
               state <= stangd_state;

            else
               state <= open_state;

            end if;

         when stangd_state =>

            if key_int_1 = '0' then
               state <= open_state;

            else
               state <= stangd_state;

            end if;

         when others =>
            state <= open_state;

      end case;
   end if;
end process;
```

```
process(state, key_int_0, key_int_1)
begin
      case state is
         when open_state=>

            if key_int_0 ='0' then
               LEDR_out <= "10";

            else
               LEDR_out <= "01";

            end if;

         when stangd_state=>

            if key_int_1 ='0' then
               LEDR_out <= "01";

            else
               LEDR_out <= "10";

            end if;
      end case;

end process;
```

3.2 MOORE - UTAN LÅS

Den här prototyp är en enkel variant, som består av fyra ingången signaler och ett utgång signal som kontrollerar funktionaliteten av den dörr konstruerade med Moore maskin.

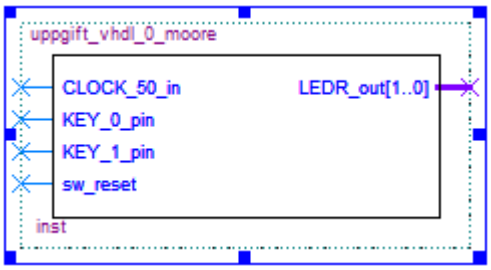
Ingången signaler

CLOCK_50_IN – (std_logic) : Den klock signal av 50MHz är kopplat till CLOCK_50.
KEY_0_PIN – (std_logic) : Ändra till nästa tillståndmaskin stäng.
KEY_1_PIN – (std_logic) : Ändra till nästa tillståndmaskin oppen.
sw_reset – (std_logic) : Reset av signalen. Active low

Utgång signal :

LEDR_out[1 downto 0] : Signal som visar ändringar av tillståndsmaskiner.

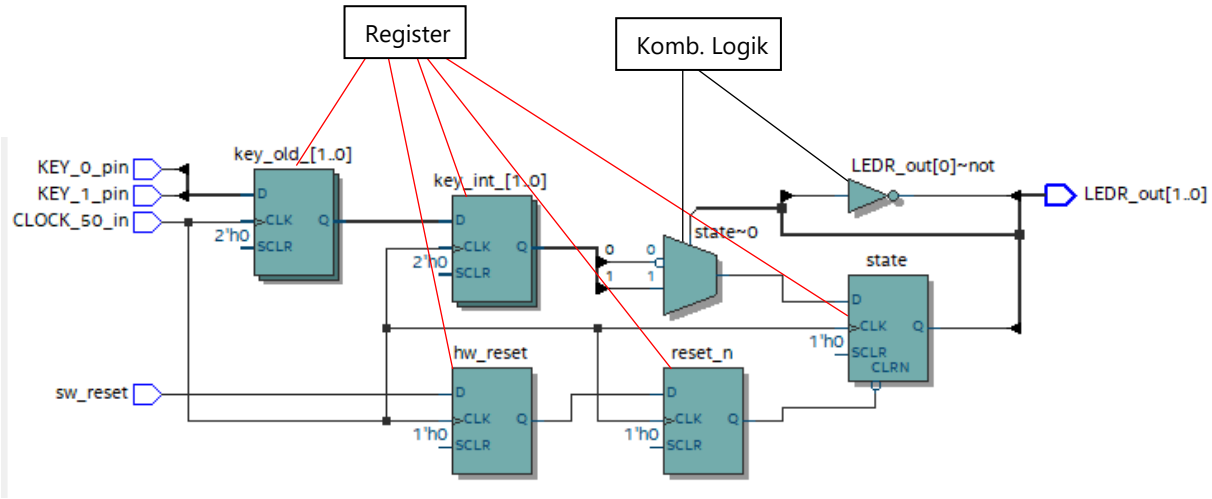
3.2.1 SYMBOL



Figur 6. Moore tillståndsmaskin symbol

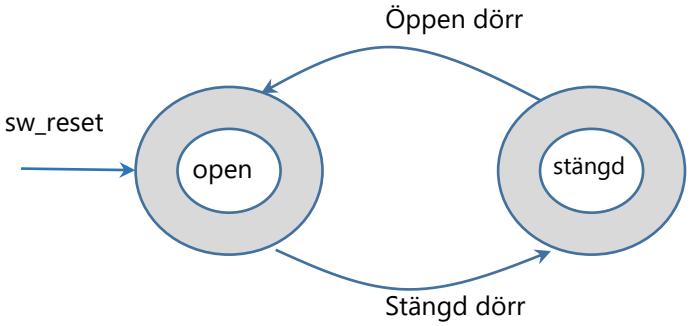
3.2.2 SYSTEM ARKITEKTUR

Den tillståndsmaskin Moore innehålla en kombinatoriska logik och register state. Det kan se i figur 7 att utgångar är enbart en funktion av tillståndet.



Figur 7. Moore system arkitektur

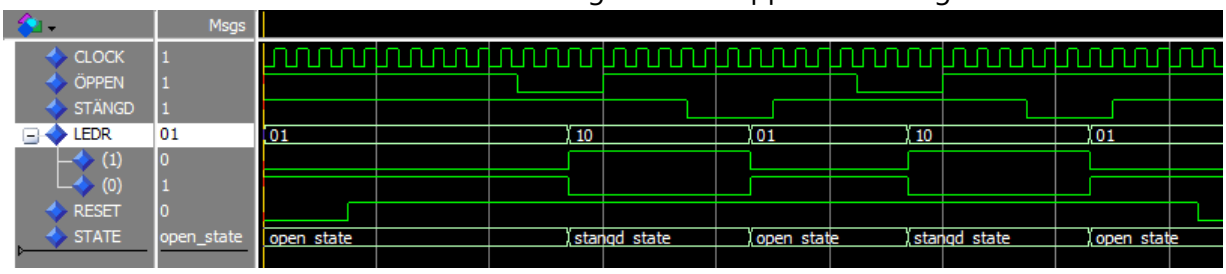
3.2.3 BESKRIVNING AV TILLSTÅNDGRAF



Figur 8. Moore tillståndgraf

3.2.4 MODELSIM

ModelSim visar när maskinen går mellan öppen och stängd tillstånd.



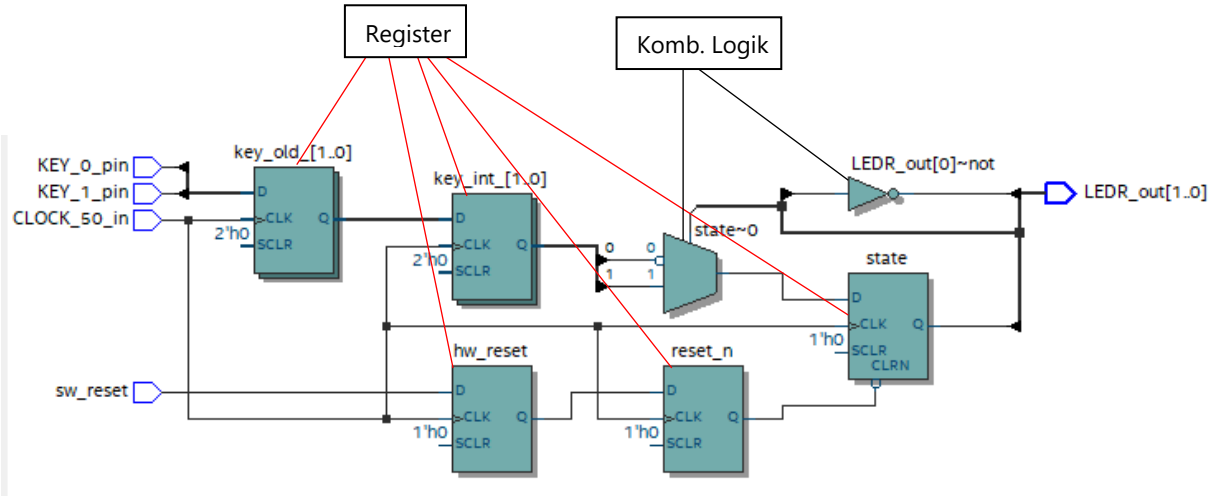
Figur 9. Pulsdiagram från ModelSim. Asynkron utgång.

3.2.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN

I figur 10 och tabell 3 visar sambandet mellan grindnivå och vhdl-koden, det står i vhdl-koden två processor som beskriver funktionaliteten av Moore maskinen.

Registers: Den delen det fungerar för att behåller nuvarande tillstånd(state)

Kombinatorisk Logic: Den delen visar hur utgången beror enbart på nuvarande tillstånd(state).



Figur 10. Grindnivå

Tabell 3. Moore Maskin. Sambandet mellan grindnivå och vhdl-koden

Resister	Kombinatorisk Logic

`process(CLOCK_50_in, reset_n)`
`begin`
`  if reset_n = '0' then`
`  -- Clear register, and clear state`
`  state <= open_state;`
`  elsif rising_edge(CLOCK_50_in) then`
`  case state is`
`  ---- Statemachine ----`
`  when open_state =>`
`  if key_int_0 ='0' then`
`  state <= stangd_state;`
`  else`
`  state <= open_state;`
`  end if;`
`  when stangd_state =>`
`  if key_int_1 = '0' then`
`  state <= open_state;`
`  else`
`  state <= stangd_state;`
`  end if;`
`  when others =>`
`  state <= open_state;`
`  end case;`
`  end if;`
`end process;`
	`process(state)`
	`begin`
	`  case state is`
	`  when open_state=>`
	`  LEDR_out <= "01";`
	`  when stangd_state=>`
	`  LEDR_out <= "10";`
	`  end case;`
	`end process;`

3.3 MEALY + MOORE - UTAN LÅS

Den här prototyp är en enkel variant, som består av fyra ingången signaler och ett utgång signal som kontrollerar funktionaliteten av den dörr konstruerade med Mealy + Moore maskin.

Ingången signaler

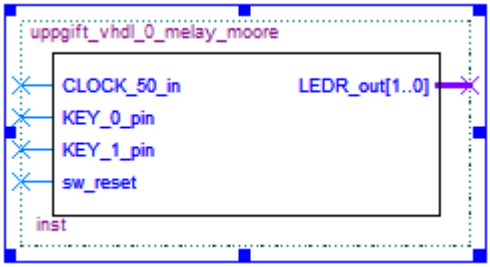
CLOCK_50_IN – (std_logic) : Den klock signal av 50MHz är kopplat till CLOCK_50.
KEY_0_PIN – (std_logic) : Ändra till nästa tillståndmaskin stäng.
KEY_1_PIN – (std_logic) : Ändra till nästa tillståndmaskin oppen.
sw_reset – (std_logic) : Reset av signalen. Active low.

Utgång signal :

LEDR_out[1 downto 0] : Signal som visar ändringar av tillståndsmaskiner.

3.3.1 SYMBOL

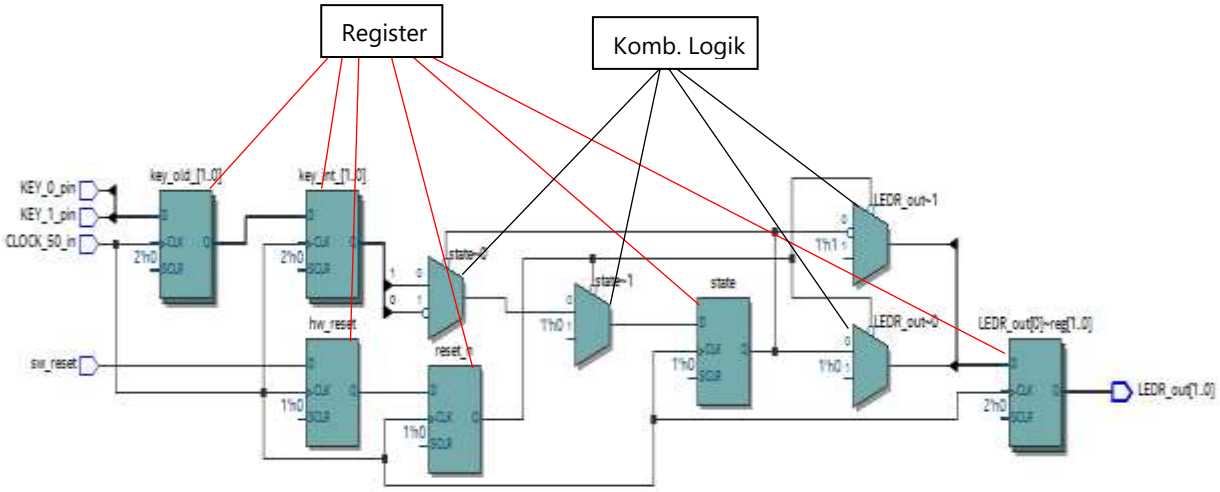
Ingångarna och utgångar av tillståndsmaskin Mealy + Moore visas i Figur 11.



Figur 11. Mealy + Moore tillståndsmaskin symbol

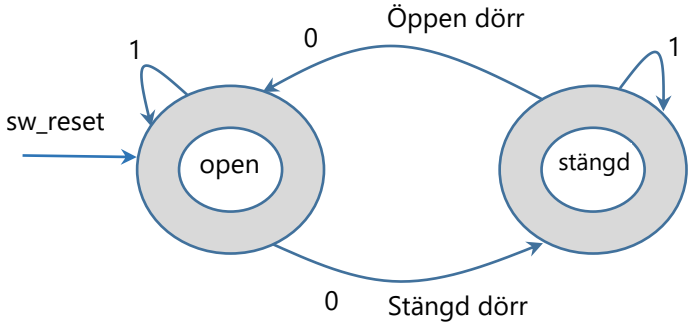
3.3.2 SYSTEM ARKITEKTUR

Den tillståndsmaskin Mealy + Moore innehålla en kombinatoriska logik och register state. Det kan se i figur 12 att när blandar Melay + Moore maskin, utgångarna(LEDR_OUT) är innehålla i registers.



Figur 12. Mealy + Moore system arkitektur

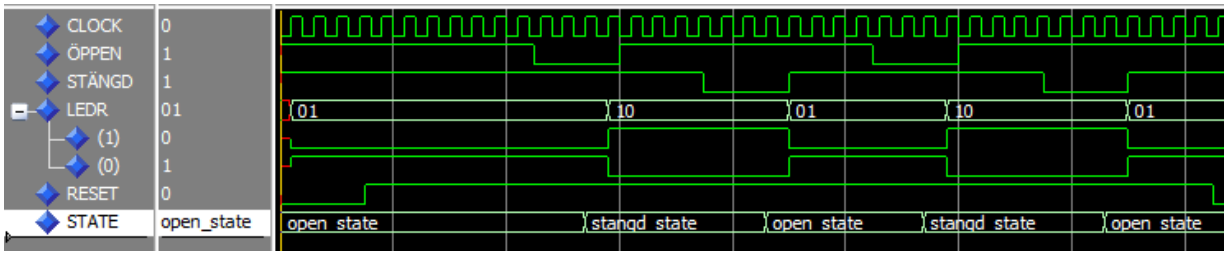
3.3.3 BESKRIVNING AV TILLSTÅNDGRAF



Figur 13. Mealy + Moore tillståndgraf

3.3.4 MODELSIM

ModelSim visar när maskinen går mellan öppen och stängd tillstånd

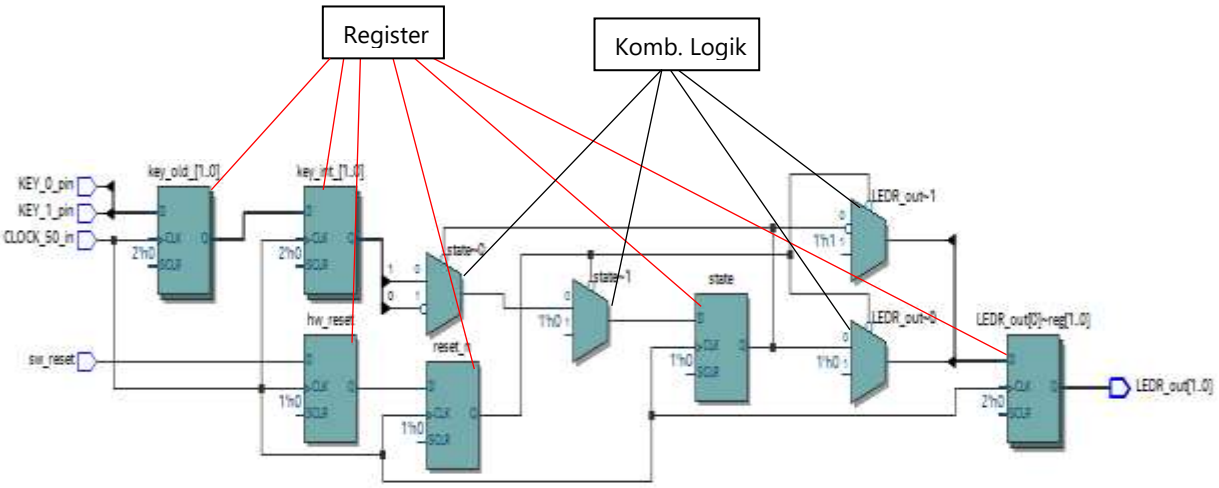


Figur 14. Pulsdiagram från ModelSim. Synkron utgång.

3.3.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN

Som visat i figur 15 och tabell 4, sambandet mellan grindnivå och vhdl-koden, det finns en processor som inkluderar funktionaliteten av Mealy + Moore maskinen samtidigt, det kan se att utgångarna(LEDR_OUT) är innehålla i registers av tillståndsmaskinen.

Registers: Den delen det fungerar för att behåller nuvarande tillstånd(state)
Kombinatorisk Logic: Den delen visar hur utgången beror enbart på nuvarande tillstånd(state).



Figur 15. Grindnivå

Tabell 4. Mealy + Moore Maskin. Sambandet mellan grindnivå och vhdl-koden

Resister - Kombinatorisk Logic	Tillståndsmaskinen

```
process(CLOCK_50_in)
begin

   if rising_edge(CLOCK_50_in) then

      if reset_n = '0' then
      -- Clear register, and clear state
         LEDR_out <= "01";
         state <= open_state;

      else

         case state is
            ---- Statemachine ----
            when open_state =>

               LEDR_out <= "01";

               if key_int_0 ='0' then
                  state <= stangd_state;
               else
                  state <= open_state;
               end if;

            when stangd_state =>

               LEDR_out <= "10";

               if key_int_1 = '0' then
                  state <= open_state;
               else
                  state <= stangd_state;
               end if;

            when others =>
               state <= open_state;

         end case;
      end if;
   end if;
end process;
```

Moore

Mealy

4. BESKRIVNINGEN AV TILLSTÅNDMASKINEN FÖR DEN AVANCERAD DÖRRSTYRNINGEN MED LÅS

Den avancerad dörrstyrningen med lås är varit konstruerat tillståndsmaskin Mealy+Moore. På nästa stegen det visar beskrivningarna av prototypen.

4.1 MEALY + MOORE - MED LÅS

Den här prototyp är en variant som består av tre ingången signaler och ett utgång signal som kontrollerar funktionaliteten av den dörr konstruerade med Mealy + Moore maskin med lås.

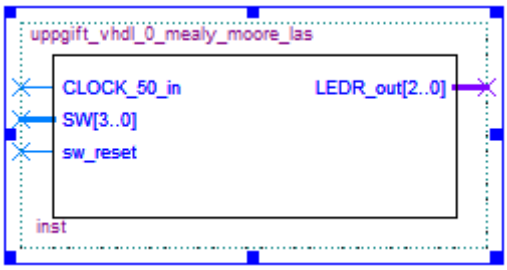
Ingången signaler

CLOCK_50_IN – (std_logic) : Den klock signal av 50MHz är kopplat till CLOCK_50.
SW – (std_logic_vector(3 downto 0)) : Switches för att ändrar tillståndsmaskiner.
sw_reset – (std_logic) : Reset av signalen. Active low.

Utgång signal :

LEDR_out[2 downto 0] : Signal som visar ändringarna av tillståndsmaskiner.

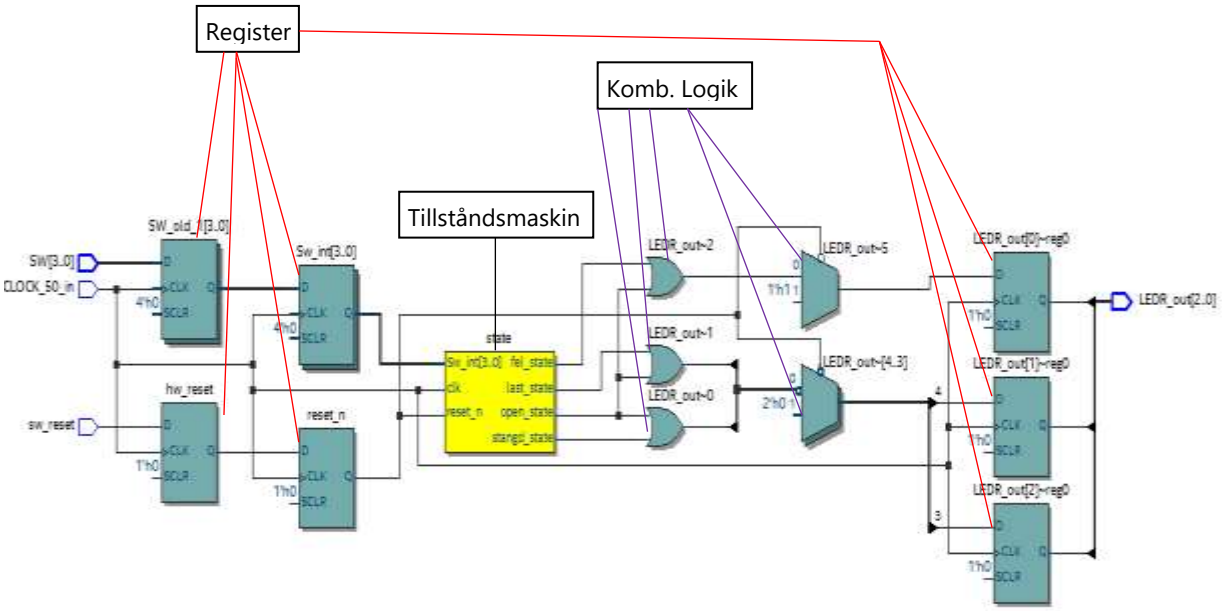
4.1.1 SYMBOL



Figur 16. Mealy + Moore med lås. Tillståndsmaskin symbol

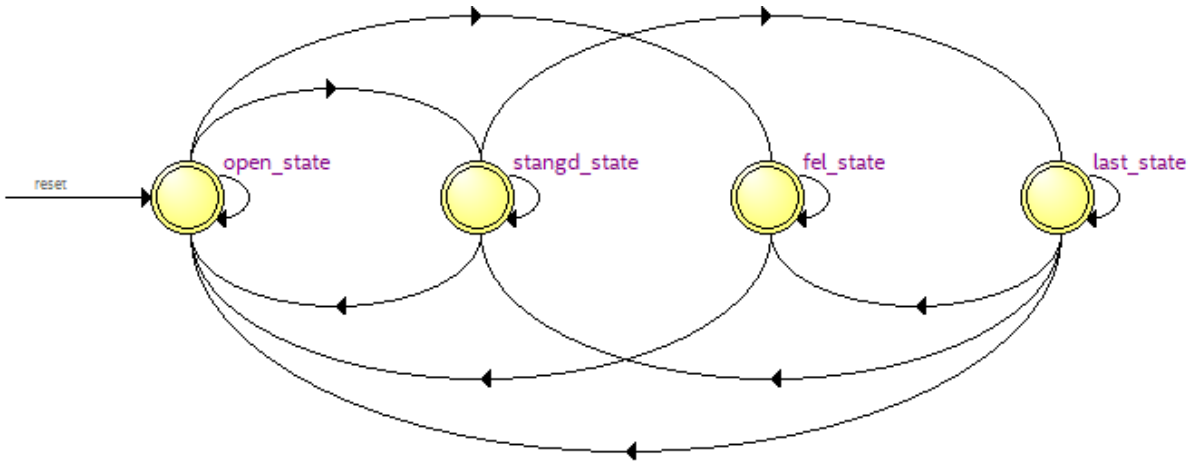
4.1.2 SYSTEM ARKITEKTUR

Den tillståndsmaskin Mealy + Moore med lås innehålla en kombinatoriska logik, register state och en gul låda som visar tillståndsmaskinen(state). Det kan se i figur 17 att utgångarna är en funktion av nuvarande tillstånd och samtliga ingångar.



Figur 17. Mealy + Moore med lås. System arkitektur

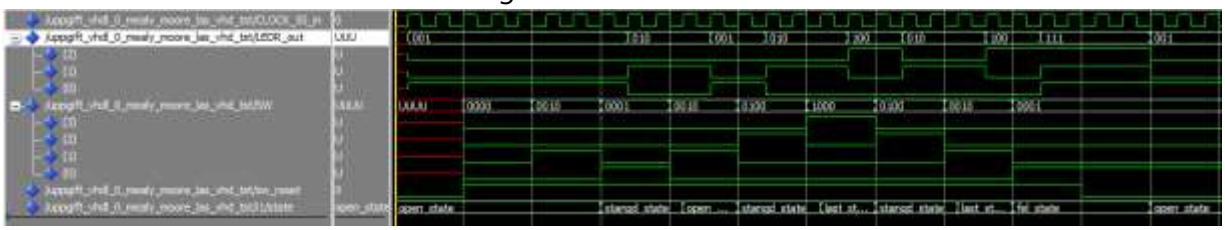
4.1.3 BESKRIVNING AV TILLSTÅNDGRAF



Figur 18. Moore tillståndgraf

4.1.4 MODELSIM

ModelSim visar när maskinen går mellan tillståndsmaskiner.

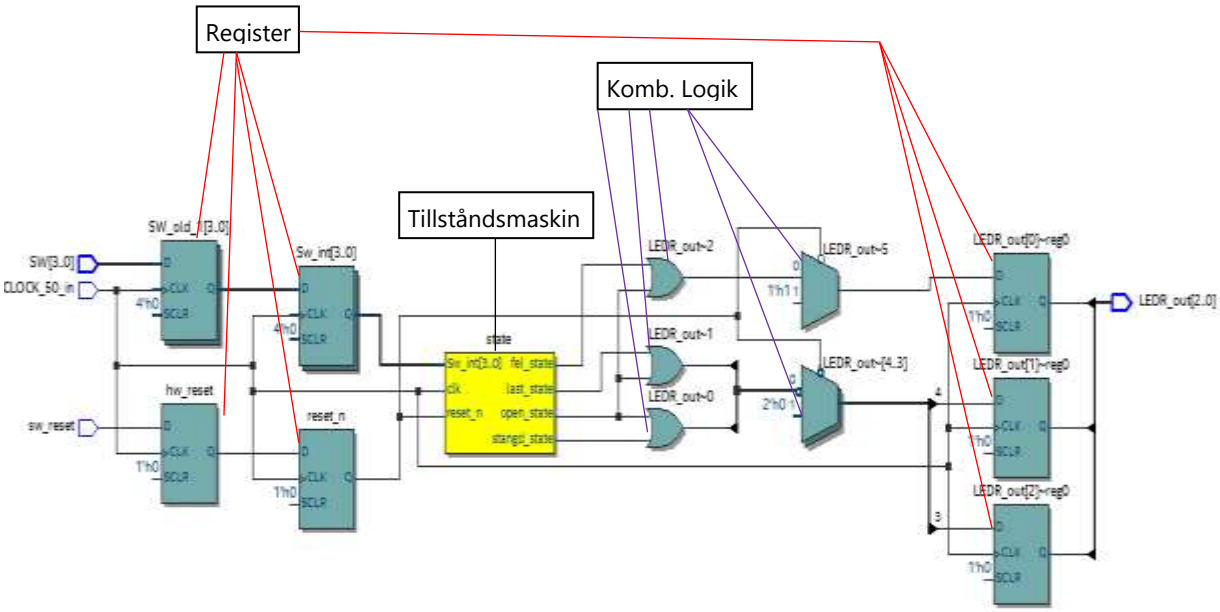


Figur 19. Pulsdiagram från ModelSim. Synkron utgång.

4.1.5 ANALYS SAMBANDET MELLAN GRINDNIVÅ OCH VHDL-KODEN

I figur 20 och tabell 5 visar sambandet mellan grindnivå och vhdl-koden, det står i vhdl-koden tre processor som beskriver funktionaliteten av Mealy + Moore maskinen med lås.

Registers: Den delen det fungerar för att behåller nuvarande tillstånd(state)
Kombinatorisk Logic: Den delen visar hur utgången beror enbart på nuvarande tillstånd(state).
State: Spara tillståndsmaskinen.



Figur 20. Mealy + Moore med lås. Grindnivå.

Tabell 5. Mealy + Moore Maskin med lås. Sambandet mellan grindnivå och vhdl-koden

Resister - Kombinatorisk Logic - State

```
process(CLOCK_50_in)
begin
    if rising_edge(CLOCK_50_in) then        -- synkron utg
        if reset_n = '0' then               -- om är reset
    -- clear register, and clear state
        LEDR_out <= "001";
        state <= open_state;                -- börjar med öpp
        else
            case state is
                ---- statemachine ----
              when open_state =>
                  LEDR_out <= "001";
                  if sw_int(1)='1' then
                     state <= stangd_state;
                  elsif sw_int(3)='1' or sw_int(2)='1' then
                     state <= fel_state;
                  else
                     state <= open_state;
                  end if;
              when stangd_state =>
                  LEDR_out <= "010";
                  if sw_int(0)='1' then
                     state <= open_state;
                  elsif sw_int(2)='1' then
                     state <= last_state;
                  else
                     state <= stangd_state;
                  end if;
             when last_state =>
                LEDR_out <= "100";
                if sw_int(3)='1' then
                   state <= stangd_state;
                elsif sw_int(1)='1' or sw_int(0)='1' then
                   state <= fel_state;
                else
                   state <= last_state;
                end if;

             when fel_state =>
                LEDR_out <= "111";
                state <= fel_state;
             when others =>
                state <= open_state;
          end case;
       end if;
    end if;
end process;
```